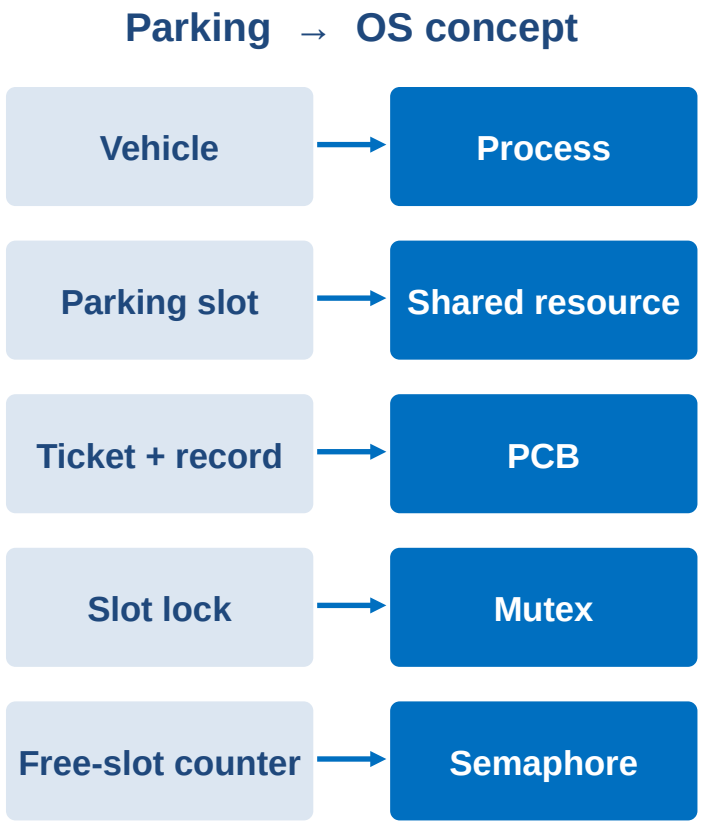
OPERATING SYSTEMS PBL

NIET Greater Noida
Autonomous Institute
Get Future Ready

Smart Parking Management System

- **Subject:** Operating Systems (CCSE0303A)
- **Institution:** NIET Greater Noida
- **Department:** CSE — Artificial Intelligence
- **Group:** 7
- **Team:** Samar Singh · Aditi Singh · Chhavi Attri
- **Faculty Mentor:** Dr. Varsha Jotwani, Associate Professor
- **Coverage:** Unit 2 and Unit 3 · SDG 11



Group 7

INTRODUCTION

NIET Greater Noida
Autonomous Institute
Get Future Ready

- **Smart Parking Management System** is an interactive, web-based parking simulation.
- It teaches Operating Systems concepts through a relatable example: vehicles act as processes, parking slots act as shared resources.
- One dashboard demonstrates process management, CPU scheduling, resource allocation, concurrency, synchronization and deadlock.
- Built with a Python HTTP server and a browser dashboard.
- Covers Unit 2 and Unit 3 of Operating Systems (CCSE0303A).

Concepts demonstrated

- Process management
- CPU scheduling
- Resource allocation
- Concurrency & synchronization
- Deadlock handling

SDG 11 — Sustainable Cities and Communities

Group 7

PARKING ANALOGY → OS CONCEPTS

NIET Greater Noida
Autonomous Institute
Get Future Ready

Parking system	Operating Systems concept
Vehicle	Process
Parking slot	Shared resource
Parking ticket and vehicle record	Process Control Block (PCB)
Waiting vehicles	Ready queue
Vehicle occupying a slot	Running process
Vehicle waiting for a free slot	Waiting (blocked) process
Gate barrier	Critical-section boundary
Slot allocation lock	Mutex
Available-slot counter	Counting semaphore
Assigning vehicles to slots	CPU scheduling and resource allocation

The analogy models OS behaviour conceptually; it does not represent CPU hardware.

Group 7

PROBLEM STATEMENT

NIET Greater Noida
Autonomous Institute
Get Future Ready

The challenge

- OS concepts such as scheduling, synchronization and deadlock are abstract when learned from theory alone.
- Textbook diagrams are static: queues, waiting and conflicts are hard to see unfolding.
- Race conditions and deadlock are difficult to reproduce and observe by hand.

Our approach

- Map every concept to a familiar parking scenario.
- Make state changes visible through interactive simulations.
- Show outcomes step by step with queues, Gantt charts and matrices.

Group 7

OBJECTIVES

NIET Greater Noida
Autonomous Institute
Get Future Ready

1. Demonstrate process management and process states
2. Visualize and compare CPU scheduling algorithms
3. Explain scheduling metrics
4. Demonstrate concurrency and synchronization
5. Simulate safe resource allocation
6. Visualize deadlock detection
7. Demonstrate classical synchronization problems

Group 7

PROPOSED SYSTEM — WORKFLOW

NIET Greater Noida
Autonomous Institute
Get Future Ready

1 Vehicle enters

New arrival in the simulation

2 Process record created

PCB: ticket + vehicle record

3 Enters a queue

Appropriate queue chosen

4 Scheduler selects

Chosen scheduling algorithm

5 Slot allocated

Suitable parking slot assigned

6 Active for duration

Simulated running time

7 Process terminates

Slot becomes available

When a slot is released

the available-slot counter increases and waiting vehicles can be scheduled.

Lifecycle: arrival → queue → scheduling → allocation → execution → termination

Group 7

SYSTEM ARCHITECTURE

NIET Greater Noida
Autonomous Institute
Get Future Ready

Browser-based frontend — HTML · CSS · JavaScript dashboard · Three.js

requests / responses

Python HTTP server — serves the dashboard and exposes API endpoints

Simulation engine

Data models
Process model
Parking-slot model

CPU scheduling engine

Concurrency & synchronization module

Banker's Algorithm & deadlock module

Classical synchronization module

Group 7

TECHNOLOGIES USED

NIET Greater Noida
Autonomous Institute
Get Future Ready

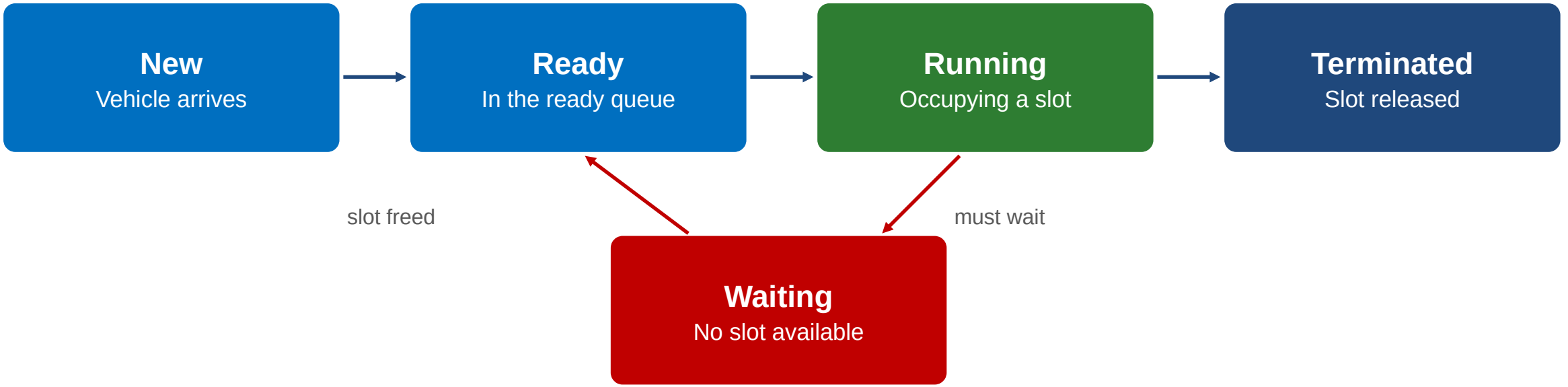
Python	HTML	CSS	JavaScript
Simulation logic and server	Dashboard structure	Dashboard styling	Dashboard interactivity

Three.js	Python HTTP server	Python threading
Graphical visualization	Serves dashboard and APIs	Locks, semaphores and other sync primitives

Group 7

PROCESS MANAGEMENT

NIET Greater Noida
Autonomous Institute
Get Future Ready



Process Control Block (PCB)

- Parking ticket + vehicle record
- Holds the process identity, state and scheduling information

Queues

- Ready queue — waiting vehicles
- Waiting queue — blocked on a slot
- Terminated — vehicles that have left

Group 7

CPU SCHEDULING ALGORITHMS

NIET Greater Noida
Autonomous Institute
Get Future Ready

Algorithm	Working principle	In the parking simulation
First-Come, First-Served (FCFS)	Runs processes in arrival order; non-preemptive	First vehicle in line gets the slot first
Shortest Job First (SJF)	Picks the shortest burst time next; non-preemptive	Shortest-stay vehicle is served first
Shortest Remaining Time First (SRTF)	Preemptive SJF: least remaining time runs	A new short-stay arrival can take over the turn
Priority (Non-Preemptive)	Highest priority is chosen; runs to completion	High-priority vehicle goes next, never interrupts
Priority (Preemptive)	A higher-priority arrival preempts the running process	High-priority vehicle can interrupt the current one
Round Robin	Fixed time quantum on a circular ready queue	Each vehicle gets a time slice, then rejoins the queue

Group 7

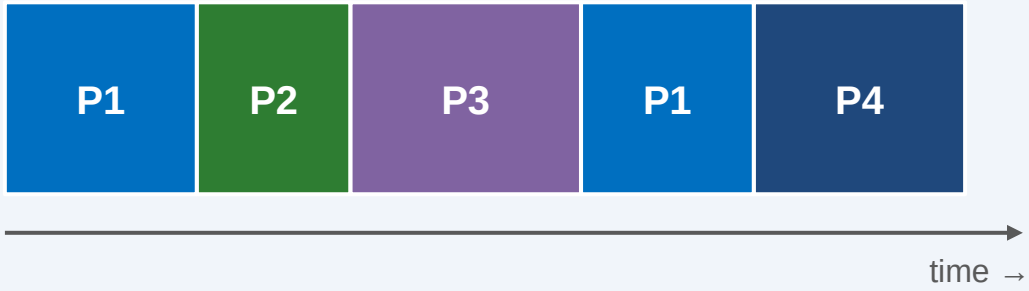
METRICS AND GANTT CHART

NIET Greater Noida
Autonomous Institute
Get Future Ready

Metrics and formulas

- **Turnaround Time** = Completion − Arrival
- **Waiting Time** = Turnaround − Burst
- **Response Time** = First Start − Arrival
- **CPU Utilization** = busy time ÷ total time × 100
- **Throughput** = processes completed ÷ total time
- **Averages** (waiting, turnaround, response) = sum over all processes ÷ number of processes

Gantt chart: execution order



- Each block is one process's turn on the CPU, in order.
- Block width represents time; gaps show idle time.
- Metrics are computed from these turns.

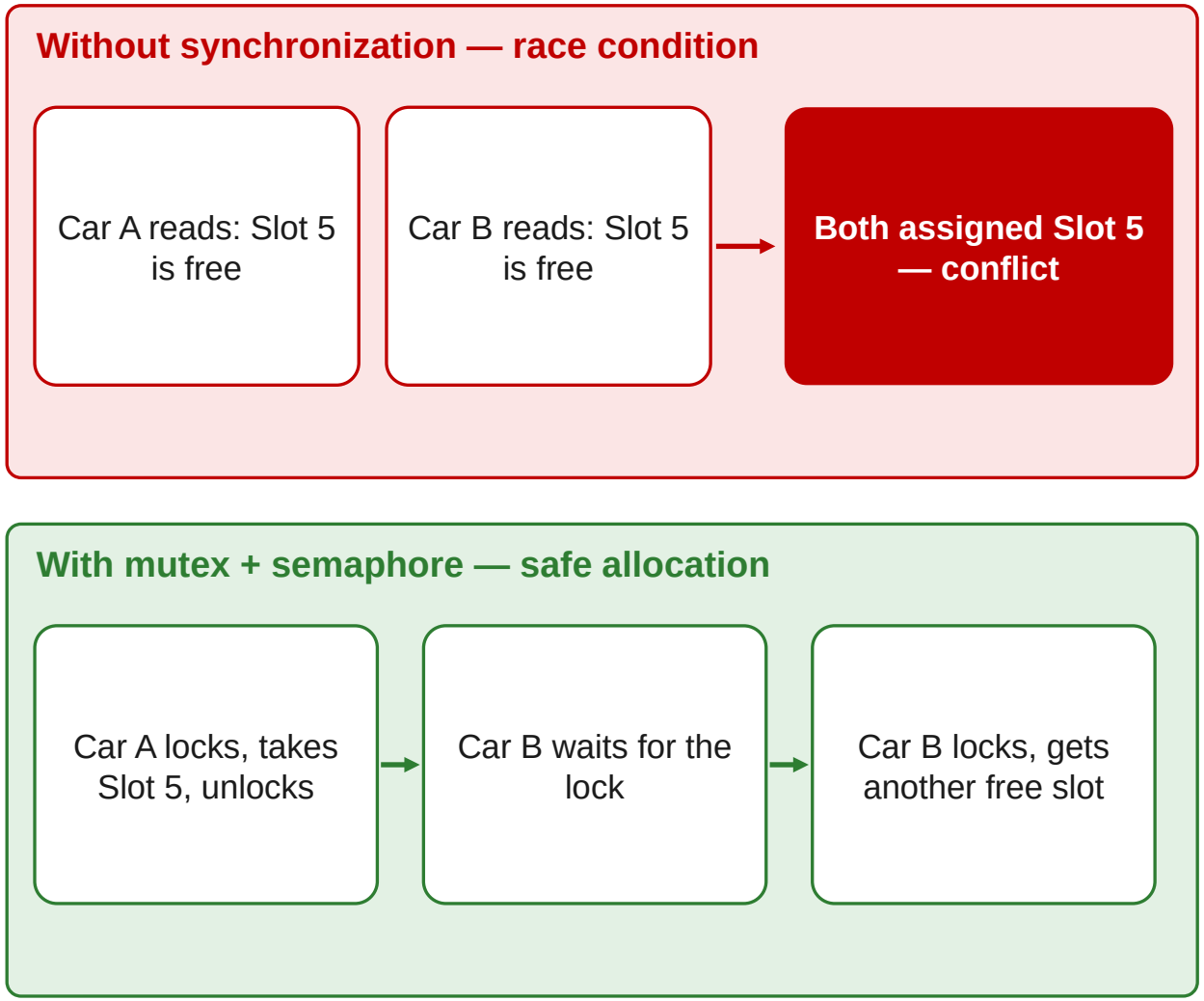
Illustrative layout only — shows how the chart is read; not measured project data.

Group 7

CONCURRENCY AND SYNCHRONIZATION

NIET Greater Noida Autonomous Institute

- **Critical section:** code that touches a shared slot
- **Race condition:** result depends on timing of simultaneous access
- **Mutex lock:** one allocator inside the critical section at a time
- **Peterson's Algorithm:** software mutual exclusion for two processes
- **Test-and-Set lock:** atomic test-and-set operation guards entry
- **Counting semaphore:** counts the available slots



Group 7

BANKER'S ALGORITHM

NIET Greater Noida
Autonomous Institute
Get Future Ready

Data structures

- **Available:** free resource units
- **Allocation:** units each process holds
- **Maximum:** most each process may request
- **Need = Maximum − Allocation**

States

- **Safe:** a safe sequence exists
- **Unsafe:** no safe sequence; deadlock possible

How a request is evaluated

Request ≤ Need?

↓

Request ≤ Available?

↓

Pretend to allocate

↓

Run safety check: safe sequence found?

↓

Yes → grant request

No → deny / wait

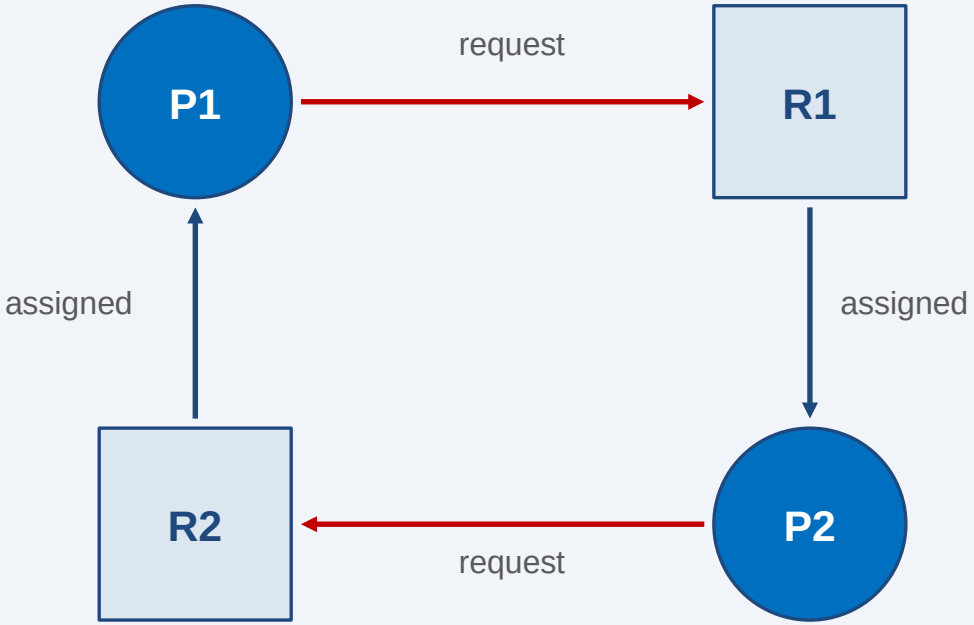
Parking view: slots are resource units; vehicles request them.

Group 7

DEADLOCK AND RAG

NIET Greater Noida Autonomous Institute

Resource Allocation Graph



A cycle in the graph signals deadlock (single-instance resources). Cycle detection finds it.

Four Coffman conditions

- **Mutual exclusion:** a slot serves one vehicle at a time
- **Hold and wait:** a vehicle holds one resource while waiting for another
- **No preemption:** a held resource cannot be forcibly taken
- **Circular wait:** a closed chain of vehicles each waits on the next
- All four must hold together for deadlock to occur.

Group 7

CLASSICAL SYNCHRONIZATION

NIET Greater Noida Autonomous Institute

Producer–Consumer

Entry camera → Waiting queue (bounded) → Slot allocator

producer → buffer → consumer

Reader–Writer

Availability signboards ↔ Parking system

readers share access; the writer updates exclusively

Sleeping Barber

Waiting chairs (limited) → Smart gatekeeper

the gatekeeper serves one vehicle at a time

Dining Philosophers

Vehicles at a roundabout → Shared road segments

circular waiting is a deadlock scenario

Group 7

DASHBOARD AND IMPLEMENTATION

NIET Greater Noida
Autonomous Institute
Get Future Ready

1 Parking Yard and PCB

Slot simulation, process records

2 CPU Scheduling and Gantt Chart

Six algorithms and metrics

3 Concurrency and Critical Section Lab

Race-condition and sync demos

4 Banker's Algorithm and Deadlock

Safe state, deadlock detection

5 Classical Synchronization Problems

Four problem simulations

6 Academic Report and Syllabus

Report and syllabus coverage

Module overview diagram — not application screenshots.

Group 7

RESULTS AND DEMONSTRATION

NIET Greater Noida
Autonomous Institute
Get Future Ready

Implemented capabilities

- ✓ Interactive parking-slot simulation
- ✓ Process queues and process states
- ✓ Six CPU scheduling algorithms
- ✓ Gantt chart and scheduling metrics
- ✓ Race-condition simulations
- ✓ Synchronization demonstrations
- ✓ Banker's Algorithm
- ✓ Deadlock detection
- ✓ Classical synchronization simulations

Implemented functionality vs. measured results: these capabilities are demonstrated live through the interactive dashboard. No benchmark or performance measurements are claimed.

Group 7

ADVANTAGES AND APPLICATIONS

NIET Greater Noida
Autonomous Institute
Get Future Ready

Educational advantages (existing)

- Real-world analogy makes abstract concepts relatable
- Queues, states, Gantt charts and matrices make behaviour visible
- Six scheduling algorithms can be compared on one dashboard
- Unit 2 and Unit 3 topics are covered in one place
- Runs in a web browser

Potential applications (proposed)

- Classroom and lab teaching aid for OS courses
- Self-study and revision tool for students
- Extension to further OS topics
- Starting point for discussing smart-parking ideas (SDG 11); conceptual, not a deployed system

Group 7

LIMITATIONS AND FUTURE SCOPE

NIET Greater Noida
Autonomous Institute
Get Future Ready

Limitations

- Educational simulation, not a production parking system
- The parking analogy simplifies real operating-system kernels
- No sensor or hardware integration
- Scope limited to Unit 2 and Unit 3 topics
- No measured performance evaluation

Future scope (proposed, not implemented)

- Additional scheduling variants
- Extension to other OS units, such as memory management
- Feedback from students to evaluate learning impact
- Export of Gantt charts and metrics as reports

Group 7

TEAM AND ACKNOWLEDGEMENT

NIET Greater Noida
Autonomous Institute
Get Future Ready

SS **Samar Singh**
Group 7 · CSE (AI)

AS **Aditi Singh**
Group 7 · CSE (AI)

CA **Chhavi Attri**
Group 7 · CSE (AI)

Contribution: presented as a collective team effort by all three members.

Acknowledgement

We sincerely thank our faculty mentor, **Dr. Varsha Jotwani, Associate Professor**, for her guidance and support throughout this Operating Systems PBL.

We also thank NIET Greater Noida and the Department of CSE — Artificial Intelligence for the opportunity.

Group 7

CONCLUSION

NIET Greater Noida Autonomous Institute



- Connects Unit 2 and Unit 3 Operating Systems theory with an interactive parking simulation.
- Covers process management, six CPU scheduling algorithms, synchronization, deadlock and classical problems.
- Makes abstract behaviour visible through queues, Gantt charts and simulations.
- An educational prototype: results are demonstrations, not benchmarks.

Group 7

REFERENCES

NIET Greater Noida
Autonomous Institute
Get Future Ready

- A. Silberschatz, P. B. Galvin, G. Gagne, Operating System Concepts, Wiley.
- A. S. Tanenbaum, H. Bos, Modern Operating Systems, Pearson.
- E. G. Coffman, M. J. Elphick, A. Shoshani, “System Deadlocks,” ACM Computing Surveys, 3(2), 1971.
- G. L. Peterson, “Myths About the Mutual Exclusion Problem,” Information Processing Letters, 12(3), 1981.
- E. W. Dijkstra, “Cooperating Sequential Processes,” 1965 (EWD123).
- Python documentation (threading) · Three.js documentation · UN Sustainable Development Goal 11.
- Smart Parking Management System — OS PBL Project Report, Group 7.

Thank You

Samar Singh · Aditi Singh · Chhavi Attri | Mentor: Dr. Varsha Jotwani